SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Performancevorhersage iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author:	Hoang-Nam Tran
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Maximilian Rieger, M.Sc.
Submission Date:	03.05.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 03.05.2026 Hoang-Nam Tran

Acknowledgments

I would like to thank my supervisor Maximilian Rieger for the ongoing valuable feedback and guidance throughout this thesis.

Abstract

Predicting query runtimes accurately is a core challenge in database systems. Traditional cost models, such as PostgreSQL's built-in estimator, rely on hand-crafted formulas and struggle with complex queries – achieving a median Q-Error of over 1000 on standard benchmarks. Learned cost models address this with machine learning, but neural network approaches introduce high prediction latency. T3 (Tuple Time Tree), a compiled decision tree model by Rieger and Neumann, resolves this trade-off: it achieves state-of-the-art accuracy on the Umbra database system at microsecond inference latency. Its authors note that T3's concepts are transferable to other database systems – a claim this thesis investigates for PostgreSQL.

We adapt T3 for PostgreSQL by converting ZeroShot benchmark execution plans into T3's pipeline representation and constructing a PostgreSQL-native feature vector. The vector captures PostgreSQL-specific signals – including planner cost estimates, planned parallelism, and operator-level features – and is used to train a LightGBM model in a leave-one-database-out setup.

Our model generalizes well across 20 unseen PostgreSQL database instances, achieving a median Q-Error below 1.33. On the Join Order Benchmark, it outperforms a prior T3-based PostgreSQL implementation and ranks 4th among all evaluated learned cost models with a median Q-Error of 2.42, demonstrating that a PostgreSQL-native feature vector is key.

However, not all T3 concepts transfer equally. The pipeline-based representation carries over well and forms an effective basis for feature extraction. Tuple-centric prediction targets, however, do not: PostgreSQL's Volcano iterator model causes per-node timing intervals to overlap across pipeline boundaries, making true exclusive per-pipeline execution times unrecoverable. As a result, per-query prediction outperforms finer-grained granularities – the opposite of T3's finding for Umbra.

This work establishes a competitive decision tree baseline for PostgreSQL runtime prediction and identifies the iterator model as the key structural barrier to a full T3 transfer.

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**

2 Theoretical Background **3**
2.1 Databases and Query Execution 3
2.1.1 Iterator-Based Execution (PostgreSQL) 4
2.1.2 Compiled Query Execution (Umbra) 5
2.1.3 Pipelines and Pipeline Breakers 6
2.2 Benchmarking and Evaluation Metrics 7
2.2.1 Benchmarking 7
2.2.2 Error Metrics 7
2.3 Performance Prediction Models 8
2.3.1 Traditional Cost Models vs. Learned Approaches 8
2.3.2 Neural Networks 9
2.3.3 Decision Trees 10
2.3.4 T3 11
2.4 Related Work 14

3 Methodology **17**
3.1 Problem Statement and Prediction Target 17
3.2 Data and Preprocessing 17
3.3 Plan Representation 18
3.4 Pipeline Decomposition 18
3.5 Feature Vector Design 19
3.6 Training 20

4 Implementation Details **22**
4.1 Plan Conversion and Pipeline Decomposition 22
4.2 Feature Vector Construction 22

5 Evaluation **26**
5.1 Generalization and Frequency Distribution 26
5.2 Comparison to Other Models 27
5.3 Ablation Study 31

6 Future Work **36**
6.1 Dedicated Benchmarking and Training Data 36
6.2 Feature Vector Extensions 37
6.3 Prediction Latency 38

7 Conclusion **39**

List of Figures **41**

List of Tables **42**

Declaration on the Use of AI-Assisted Tools **43**

Bibliography **44**

1 Introduction

Predicting the runtime of a database query before executing it is a fundamental problem in database systems research. Performance predictions enable query scheduling, resource scaling, and query optimization [1]. A query optimizer, for instance, compares alternative execution plans and selects the one it estimates to be fastest. This makes the quality of runtime predictions directly relevant to the efficiency of query execution.

Traditional cost models, such as the one built into PostgreSQL, rely on hand-crafted formulas and statistical summaries of the stored data [2]. However, these models struggle with complex queries.

Learned cost models address this by using machine learning to capture system behavior from execution traces. Recent work shows that they substantially outperform hand-crafted models [3]. Neural network approaches achieve high accuracy but suffer from high prediction latency [4].

Rieger and Neumann address this trade-off with T3 (Tuple Time Tree) [1], a compiled gradient-boosted decision tree model for the Umbra database system. T3 achieves state-of-the-art accuracy while being orders of magnitude faster than neural network models. It relies on two core ideas: decomposing query plans into pipelines and predicting the per-tuple (per-row) execution time for each pipeline independently. Rieger and Neumann explicitly note that, while T3 is implemented for Umbra, its concepts are transferable to other database systems [1].

PostgreSQL is one of the most widely deployed open-source relational database systems [5]. It uses the Volcano iterator model [6] rather than Umbra's compiled execution model [7], and exposes different plan information [8]. This raises the question of whether T3's pipeline-based approach can effectively transfer to PostgreSQL.

Research Question. *How accurately can a decision tree model predict query runtimes on an iterator-based database system such as PostgreSQL? Which of T3's key concepts – pipeline-based representation and tuple-centric prediction targets – transfer effectively to this setting? How competitive is the result against existing learned cost models?*

To answer this, we adapt T3 for PostgreSQL using the public ZeroShot benchmark data [3]. We convert parsed PostgreSQL execution plans into T3's pipeline representation and construct a PostgreSQL-native feature vector. We then train and evaluate

a LightGBM model [9] on a leave-one-database-out split to test generalization across unseen database instances.

Our results show that the adapted model generalizes well. Furthermore, on the Join Order Benchmark, our model outperforms an earlier T3-based PostgreSQL implementation by Wehrstein et al. [10], demonstrating the importance of PostgreSQL-native features.

Compared to a broader set of learned cost models, our model ranks among the top performers, achieving a median Q-Error of 2.42. However, it still falls behind state-of-the-art neural network models and T3 (Umbra).

A key finding is that PostgreSQL's iterator-based execution model makes exact pipeline timing extraction infeasible, which limits the benefit of T3's per-pipeline and per-tuple prediction strategies. Per-query prediction works best in our PostgreSQL setting.

2 Theoretical Background

2.1 Databases and Query Execution

A database management system (DBMS) is software that stores, organizes, and retrieves structured data. Users interact with such systems primarily through Structured Query Language (SQL), a declarative language that lets users specify what data they want rather than how to retrieve it [11]. For example, a simple SQL query might select all orders placed by a particular customer by joining an orders table with a customers table and ltering by a customer name. Because SQL describes the desired result and not the retrieval procedure, the DBMS is free to choose any execution strategy that produces the correct answer.

When a DBMS receives an SQL query, it processes the query in three successive phases [11]:

1. Parsing and translation. The SQL text is checked for syntactic correctness and translated into an internal relational-algebra expression. During this step, the parser also expands any view references — a view is a named, stored query that acts as a virtual table — by substituting them with their underlying relational-algebra de nition.

2. Optimization. The optimizer takes the relational-algebra expression and uses statistical summaries of the stored data to evaluate many possible execution plans. A plan is a tree of physical operators — such as table scans, joins, sorts, and aggregations — along with instructions for how each operator should be executed (e.g., whether to use hash tables or nested loops).

   One key input to the optimizer is the cardinality of each operator: the number of rows (tuples) it is expected to produce or consume. Since the query has not yet been executed, the optimizer relies on estimated cardinalities, derived from table sizes and column value distributions. These estimates directly in uence which plan the optimizer considers cheapest and therefore selects.

   The actual cardinality — the true number of rows processed at runtime — only becomes known after execution. Estimated and actual cardinalities can differ substantially, which is a recurring challenge in performance prediction [12].

Therefore, we start by summarizing different information categories per pipeline with sum, max, and average statistics. First, the `pg_card_...` fields capture output cardinality summaries.

Insights about PostgreSQL-specific cost, parallelism, and width features are captured in the `pg_est_cost_...`, `pg_est_startup_...`, `pg_workers_planned_sum` and `pg_est_width_avg` fields.

Further, the `pg_num_(scan/join/sort/etc.)` family counts operator classes.

Moreover, the `pg_scan_has_filter` and `pg_filter_(and/or/compare/like/in)` counters enumerate scan predicate attributes.

Finally, pipeline-context features describe structure using `pg_pipeline_num_ops` for instance.

2. **Operator-level block.** For every operator that occurs in `parsed_plans`, we assign a slice in the feature vector. If multiple operators of the same type occur in the pipeline, we sum the features over all occurences. In Figure **??**, the excerpt of the operator-type block for the `Nested_Loop join` operator is depicted.

   Depending on the operator type, we include different features. This feature set is derived from T3's Umbra implementation [1]. For example, a join would include different features than a scan, like in the Umbra/T3 approach.

   Regarding the calculations of the features, we follow the Umbra/T3 implementation where adequate. A deviation would be, e. g., how expression percentages are formed due to the different nature of PostgreSQL's filter predicates (CITE).

   Finally, we include the operator count "`const`

Pipeline-level block (excerpt; first 12 of 23 features)	
`pg_card_sum`	6541.0
`pg_card_max`	6537.0
`pg_est_cost_sum`	16882.95
`pg_est_cost_max`	16877.3
`pg_est_startup_sum`	125.82
`pg_workers_planned_sum`	4.0
`pg_est_width_avg`	10.0
`pg_num_scan`	1.0
`pg_num_join`	1.0
`pg_scan_has_filter`	0.0
`pg_filter_compare_count`	0.0
`pg_pipeline_num_ops`	2.0
Operator-type block (excerpt; Nested_Loop slice; 130 entries in full)	
`Nested_Loop_const`	1.0
`Nested_Loop_in_percentage`	0.11473152822395594
`Nested_Loop_right_card`	98378.0
`Nested_Loop_out_percentage`	0.008260670032124828
(remaining operator-type entries omitted)	

Figure 4.2: Excerpts of the pipeline-level block (first twelve features) and of the `Nested_Loop` operator entries in the operator-type block. They are from one pipeline of query `index_workload_100k_s2_c8220_890` on the TPC-H

prediction models [4].

Training on estimated instead of exact cardinalities partially compensates for this effect. The model trained and evaluated on estimated cardinalities achieves a lower average Q-Error than the model trained on exact cardinalities but evaluated on estimated ones. Thus, this demonstrates that the model can somewhat learn to account for the systematic biases of the cardinality estimator [12]. However, the p90 Q-Error stays at a similar, high level in both estimated-cardinality settings, indicating that a small number of heavy outliers remain.

We conclude that incorrect cardinality estimates distort performance prediction accuracy substantially. Cardinality estimation remains an important and unsolved problem [12], and improvements in cardinality estimation would directly benefit performance prediction.

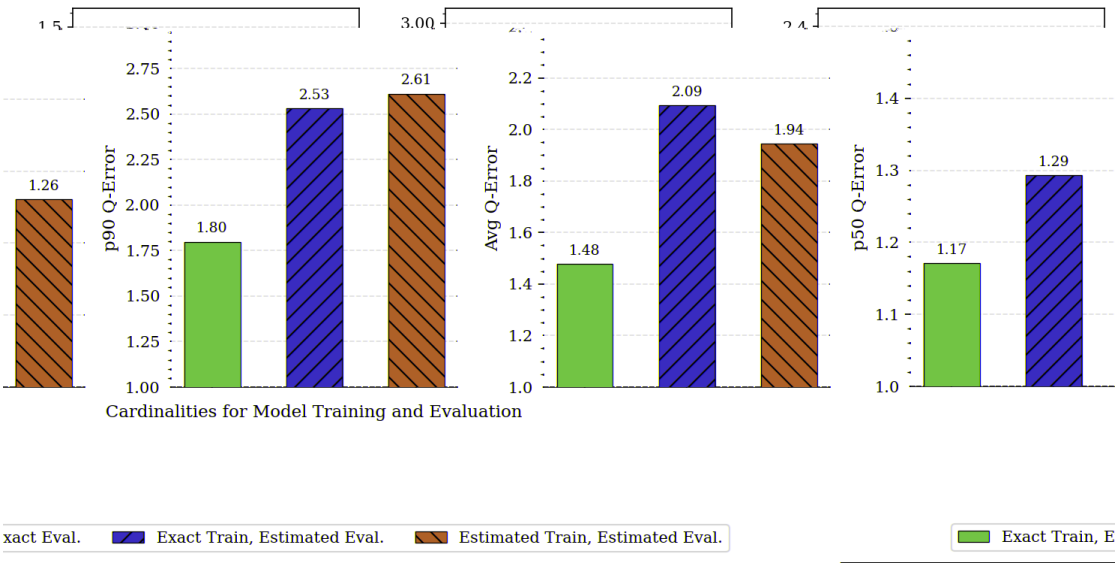

Figure 5.6: Q-Error (p50, p90, and average) over all queries under three cardinality settings: trained and evaluated on exact cardinalities, trained on exact but evaluated on estimated cardinalities, and trained and evaluated on estimated cardinalities.

Per-Query vs. Per-Pipeline vs. Per-Tuple Targets. Figure **??** compares Q-Error across three prediction granularities: per-query, per-pipeline, and per-tuple. This directly copies the evaluation of Rieger and Neumann [1], who find that finer granularities consistently improve accuracy —- with per-tuple being best and per-query being worst.

We observe the opposite: the per-query model achieves the lowest Q-Error across all three metrics, while per-pipeline and per-tuple models perform noticeably worse.

The core issue is the reliability of the per-pipeline execution times that can be extracted from PostgreSQL's output in the ZeroShot dataset [8]. In Umbra, pipeline breakers require full materialization before any downstream pipeline can begin scanning their output [1], making pipelines sequential execution units whose individual durations sum to the total query time.

Unlike Umbra, PostgreSQL query plans do not expose pipeline execution times directly. Instead, they must be derived from the per-node durations recorded by `EXPLAIN ANALYZE` [8].

This derivation is fundamentally limited by PostgreSQL's Volcano iterator model [6], which does not execute pipelines as separate sequential units. Nodes from different pipelines execute interleaved rather than one after the other.

For example, a hash join operator (belonging to pipeline P1) builds its hash table by calling `next()` on its input table scan (belonging to pipeline P0) one tuple at a time. The join and the scan therefore execute interleaved over the same wall-clock period: the join is active from its first `next()` call to the moment the hash table is complete, and the scan is active during every individual tuple fetch within that same period.

As a result, `EXPLAIN ANALYZE` reports two cumulative wall-clock timestamps per node: the time from query start to when the node produced its first output row, and the time to when it produced its last row. These are wall-clock durations, not exclusive execution times – each node's interval includes the time it spent waiting for its children. Consequently, both the join (P1) and the scan (P0) receive overlapping time intervals, even though they belong to different derived pipelines.

To derive pipeline times from these overlapping intervals, we first compute each pipeline's span as the range from the earliest node start to the latest node stop within that pipeline. We then assign each pipeline a proportional share of the actual query time, scaled by its span relative to the total observed plan span. A correction factor is applied so that the pipeline times sum exactly to the actual query time.

However, this proportional allocation is a heuristic. Because the node intervals overlap, the raw spans sum to more than the total observed span — and the correction factor compensates arithmetically rather than recovering the true exclusive execution time of each pipeline. The individual pipeline labels therefore remain imprecise, introducing noise into the training targets.

For instance, in plan 2 of `tpch/workload_100k_s1_c8220`, pipeline P0 (table scan) is recorded from $t = 946$ms to $t = 2{,}156$ms, giving a span of 1,210ms. Pipeline P1 (table scan, join) is recorded from $t = 0$ms to $t = 2{,}185$ms, giving a span of 2,185ms. P0 is entirely nested inside P1's time window, so the raw spans already sum to 3,395ms against an actual query time of 2,239ms. The correction factor of rescales both pipeline

times so their sum is correct on a query level, but it cannot determine how much of the 2,239ms truly belongs to P0 versus P1.

The per-tuple approach, which divides the pipeline runtime into per-tuple runtimes, is additionally weakened by a difference in dataset characteristics. In T3, the query execution time span is larger than 10^{-5} s to 10 s [1]. Thus, Rieger and Neumann identify this wide range as the primary motivation for the per-tuple approach. Decision trees have a limited number of discrete leaf values and struggle to predict accurately over such a large range. Hence, normalizing by tuple count compresses the prediction target and improves accuracy [1]. However, in our PostgreSQL dataset, query execution times span a much narrower range, as visible in Figure **??**.

Subsequently, with a narrower range and per-pipeline targets that are already noisy, the per-tuple decomposition provides no accuracy benefit and introduces additional label inaccuracies instead.

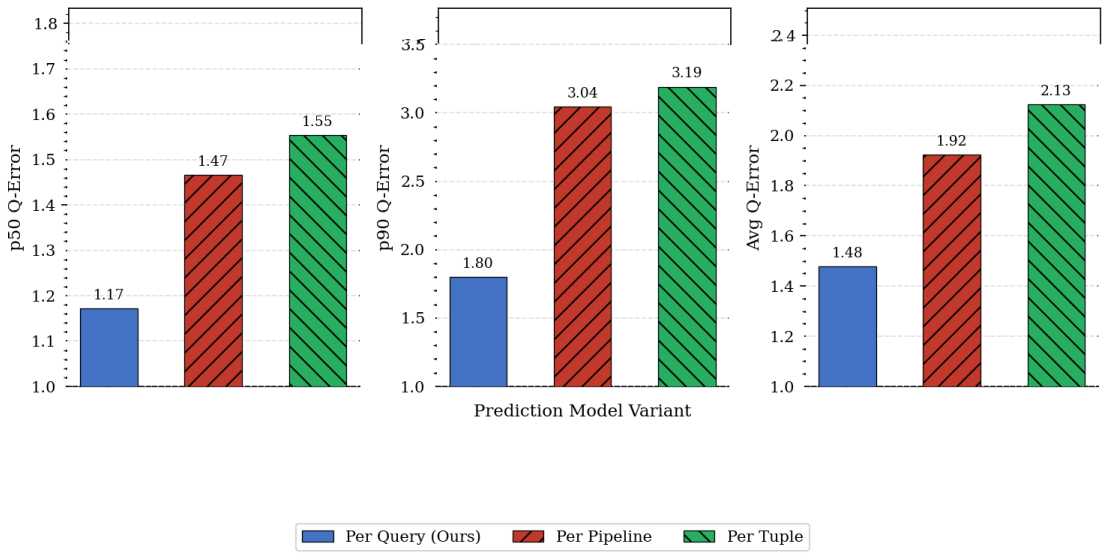

Figure 5.7: Q-Error (p50, p90, and average) for three prediction granularities over all queries: per-query, per-pipeline, and per-tuple.

5 Evaluation

6 Future Work

6.1 Dedicated Benchmarking and Training Data

The most significant limitation of this work is the training data. We rely entirely on the ZeroShot benchmark data, which was designed to enable cost estimation that generalizes to unseen database instances, rather than to cover the full spectrum of PostgreSQL query behavior [3].

The ZeroShot training queries follow a narrow pattern of selective table scans, equi-joins, and a final aggregation to a single tuple [3]. In contrast, T3 generates training data from 16 distinct query structures, explicitly including complex selection predicates and all combinations of filters, joins, aggregations, and sorts [1]. These query types are largely absent from the ZeroShot training data, which limits how well our model can generalize to more diverse workloads.

The JOB comparison was restricted to 77 of the 113 benchmark queries, because the remaining queries exceed the 30 s cutoff of the ZeroShot data [3]. Rieger and Neumann evaluated T3 on all 113 queries [1], making a direct like-for-like comparison on the full JOB benchmark impossible.

Conseqeutively, building a dedicated PostgreSQL benchmark in the style of T3's Umbra benchmark would close the query diversity gap and restore a full JOB comparison.

The runtime distribution of the ZeroShot data is also considerably narrower than T3's Umbra training data [1]. A dedicated benchmark covering a wider runtime range would allow the per-tuple prediction approach to be re-evaluated, since T3's per-tuple normalization is specifically motivated by such a wide range [1] and our ablation study could not test this under comparable conditions (Section **??**).

Moreover, a data gap is the absence of tablescan input cardinalities in the `parsed_plans`. Our implementation always sets this value to 1, and the model compensates via the

6.2 Feature Vector Extensions

Finer-grained pipeline features. Currently, the pipeline-level block of our feature vector aggregates features across all nodes in a pipeline. For example, `workers_planned` is summed over the pipeline. A natural extension is to expand the pipeline block with per-node entries, at the cost of a larger feature vector. This would let the model distinguish, for example, a pipeline where only one node is parallelized from one where all nodes run in parallel.

Purely query-level feature vector. Per-query prediction outperforms per-pipeline and per-tuple prediction in our setting. Thererefore, a future direction is to design the feature vector directly at the query level, without the intermediate pipeline decomposition. This would further sidestep the PostgreSQL pipeline timing problem identified in the ablation study (Section **??**), while also simplifying the model pipeline.

Joint pipeline and query context for pipeline-level prediction. For the per-pipeline prediction approach, a possible extension is to provide two feature vectors per pipeline: one capturing only the local pipeline features, and one summarizing the full query context. This gives the model information about how a pipeline relates to the rest of the query, which may improve per-pipeline runtime prediction.

Impact of PostgreSQL cost estimates. We include PostgreSQL planner cost estimates (`est_cost`, `est_startup_cost`) as features. However, the ablation study confirms that the PostgreSQL cost estimator is highly inaccurate for complex queries (median Q-Error above 1000, see Table **??**). A targeted ablation study that removes or replaces these planner-cost features could determine whether they help or hurt prediction accuracy.

Mitigating the pipeline timing issue. The pipeline timing problem identified in the ablation study arises because PostgreSQL's `EXPLAIN ANALYZE` reports cumulative wall-clock intervals per node, not exclusive execution times [31]. A node's reported interval includes time spent waiting for its children, which causes intervals to overlap across pipeline boundaries.

Hence, a thorough long-term approach would be to modify PostgreSQL's executor instrumentation to track exclusive CPU time per node directly. This would require extending the executor's timing hooks to subtract child (loc)-265(g)-2p1(dir8.l4a241s45091 Tf3ck)-31-29 p

6.3 Prediction Latency

Since both our model and T3 use LightGBM gradient-boosted decision trees, prediction should be fast in practice. Rieger and Neumann note that T3's low prediction latency is a key practical advantage [1]. Future work should benchmark the prediction latency of our PostgreSQL model to confirm its performance, and to compare it directly against T3's reported latency [1]. A concrete step toward this would be to compile the trained

7 Conclusion

This thesis investigated how accurately a decision tree model can predict query runtimes on an iterator-based database system such as PostgreSQL. Furthermore, it analyzed which of T3's key concepts – pipeline-based representation and tuple-centric prediction targets – transfer effectively, and how the result compares to existing learned cost models.

We adapted T3 for PostgreSQL by converting parsed ZeroShot execution plans into T3's pipeline representation and constructing a PostgreSQL-native feature vector. The feature vector includes PostgreSQL-specific signals such as planner cost estimates and planned parallelism alongside operator-level features derived from T3's Umbra design. We trained a LightGBM model with a leave-one-database-out split, mirroring both T3's and ZeroShot's generalization evaluation.

The model generalizes reliably across unseen PostgreSQL database instances, with a median Q-Error below 1.33 across all 20 ZeroShot databases. On the Join Order Benchmark, it clearly outperforms the T3-based PostgreSQL implementation of Wehrstein et al. [10]. Consequently, this confirms that a PostgreSQL-native feature vector is beneficial. Among all learned cost models evaluated on JOB, our model ranks 4th with a median Q-Error of 2.42. This is well ahead of most learned approaches and far better than PostgreSQL's built-in estimator at over 1000. However, it falls behind SOTA neural network models such as ZeroShot (1.58) and DACE (1.91), and behind T3 (Umbra).

Regarding T3's key concepts, the pipeline-based representation transfers well. It provides the structural basis for feature extraction and produces a compact, informative feature vector.

However, tuple-centric prediction targets do not transfer. PostgreSQL's iterator model causes per-node timing intervals to overlap across pipeline boundaries, so true exclusive execution times per pipeline are unrecoverable. Per-query prediction therefore works best – the opposite of T3's finding for Umbra.

Finally, a promising path to closing the gap to T3 (Umbra) and other SOTA models is a dedicated PostgreSQL benchmark covering a wider range of query types and runtimes. Subsequently, the per-tuple approach could be re-evaluated, especially with

compilation using the

List of Figures

2.1 T3 prediction process 12

4.1 Top-node cutout with Hash Join 23
4.2 Excerpt of a PostgreSQL-native pipeline feature vector 25

5.1 Q-Error across Database Instances 27
5.2 Q-Error Frequency Distribution 28

Declaration on the Use of AI-Assisted Tools

AI tools were used in two capacities during this project.

For implementation, Claude was used as a coding assistant – for example, to navigate the codebase, generate boilerplate, and debug. Generated code was reviewed, tested, and adapted to the specific requirements of this project.

Regarding writing, Claude was used as an assistant for drafting, structuring, and

Bibliography

[1] M. Rieger and T. Neumann. "T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees." In: *Proc. ACM Manag. Data* 3.3 (June 2025). doi: 10.1145/3725364.

[2] *PostgreSQL 16 Documentation: How the Planner Uses Statistics*. The PostgreSQL Global Development Group. 2024.

[3] B. Hilprecht and C. Binnig. "Zero-shot cost models for out-of-the-box learned cost prediction." In: *Proc. VLDB Endow.* 15.11 (July 2022), pp. 2361–2374. issn: 2150-8097. doi: 10.14778/3551793.3551799.

[4] Z. Wu, R. Marcus, Z. Liu, P. Negi, V. Nathan, P. Pfeil, G. Saxena, M. Rahman, B. Narayanaswamy, and T. Kraska. "Stage: Query Execution Time Prediction in Amazon Redshift." In: *Companion of the 2024 International Conference on Management of Data (SIGMOD)*. ACM, 2024, pp. 280–294. doi: 10.1145/3626246.3653396.

[5] *DB-Engines Ranking*. DB-Engines. 2025.

[6] G. Graefe. "Volcano—An Extensible and Parallel Query Evaluation System." In: *IEEE Transactions on Knowledge and Data Engineering* 6.1 (1994), pp. 120–135. doi: 10.1109/69.273032.

[7] T. Neumann and M. J. Freitag. "Umbra: A Disk-Based System with In-Memory Performance." In: *10th Conference on Innovative Data Systems Research (CIDR)*. 2020.

[8] *PostgreSQL 16 Documentation: The Path of a Query*. The PostgreSQL Global Development Group. 2024.

[9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[10] J. Wehrstein, T. Eckmann, R. Heinrich, and C. Binnig. "JOB-Complex: A Challenging Benchmark for Traditional & Learned Query Optimization." In: *arXiv preprint arXiv:2507.07471* (2025).

[11] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts*. 7th ed. McGraw-Hill Education, 2019.

[12] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. A. Boncz, A. Kemper, and T. Neumann. "Query optimization through the looking glass, and what we found running the Join Order Benchmark." In: *The VLDB Journal* 27.5 (2018), pp. 643–668. doi: 10.1007/s00778-017-0480-7.

[13] T. Neumann. "Efficiently Compiling Efficient Query Plans for Modern Hardware." In: *Proceedings of the VLDB Endowment* 4.9 (2011), pp. 539–550. doi: 10.14778/2002938.2002940.

[14] G. Moerkotte, T. Neumann, and G. Steidl. "Preventing bad plans by bounding the impact of cardinality estimation errors." In: *Proc. VLDB Endow.* 2.1 (2009), pp. 982–993. doi: 10.14778/1687627.1687738.

[15] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[16] J. Sun and G. Li. "An End-to-End Learning-based Cost Estimator." In: *Proc. VLDB Endow.* 13.3 (2019), pp. 307–319. doi: 10.14778/3368289.3368296.

[17] R. Marcus and O. Papaemmanouil. "Plan-Structured Deep Neural Network Models for Query Performance Prediction." In: *Proc. VLDB Endow.* 12.11 (2019), pp. 1733–1746. doi: 10.14778/3342263.3342646.

[18] Y. Zhao, G. Cong, J. Shi, and C. Miao. "QueryFormer: A Tree Transformer Model for Query Plan Representation." In: *Proc. VLDB Endow.* 15.8 (2022), pp. 1658–1670. doi: 10.14778/3529337.3529349.

[19] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, 1984.

[20] J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine." In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. doi: 10.1214/aos/1013203451.

[21] S. Listgarten and M.-A. Neimat. "Cost Model Development for a Main Memory Database System." In: *Real-Time Database Systems: Issues and Applications*. Ed. by A. Bestavros, K.-J. Lin, and S. H. Son. Boston, MA: Springer US, 1997, pp. 139–162. doi: 10.1007/978-1-4615-6277-3_8.

[22] S. Manegold, P. A. Boncz, and M. L. Kersten. "Generic Database Cost Models for Hierarchical Memory Systems." In: *Proceedings of the 28th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2002, pp. 191–202.

[23] C. Gupta, A. Mehta, and U. Dayal. "PQR: Predicting Query Execution Times for Autonomous Workload Management." In: *Proceedings of the 5th International Conference on Autonomic Computing (ICAC)*. IEEE Computer Society, 2008, pp. 13–22. doi: 10.1109/ICAC.2008.12.

[24] A. Ganapathi, H. A. Kuno, U. Dayal, J. L. Wiener, A. Fox, M. I. Jordan, and D. A. Patterson. "Predicting Multiple Metrics for Queries: Better Decisions Enabled by Machine Learning." In: *Proceedings of the 25th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2009, pp. 592–603. doi: 10.1109/ICDE.2009.130.

[25] A. Kipf, T. Kipf, B. Radke, V. Leis, P. A. Boncz, and A. Kemper. "Learned Cardinalities: Estimating Correlated Joins with Deep Learning." In: *9th Conference on Innovative Data Systems Research (CIDR)*. 2019.

[26] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul. "Neo: A Learned Query Optimizer." In: *Proc. VLDB Endow.* 12.11 (2019), pp. 1705–1718. doi: 10.14778/3342263.3342644.

[27] Z. Liang, X. Chen, Y. Xia, R. Ye, H. Chen, J. Xie, and K. Zheng. "DACE: A Database-Agnostic Cost Estimator." In: *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 2024, pp. 4925–4937. doi: 10.1109/ICDE60146.2024.00374.

[28] Y. Zhao, S. Mo, and G. Cong. "TATA: An Efficient Framework for Task Transfer in Query Plan Representation." In: *Proc. VLDB Endow.* 19.3 (2025), pp. 413–425. doi: 10.14778/3778092.3778102.

[29] R. Heinrich, M. Luthra, J. Wehrstein, H. Kornmayer, and C. Binnig. "How Good are Learned Cost Models, Really? Insights from Query Optimization Tasks." In: *Proc. ACM Manag. Data* (2025). doi: 10.1145/3725309.

[30]